

Meeting Assistant - AI Coding Instructions

This repository contains a meeting transcription and analysis platform. Follow these guidelines to work effectively in this codebase.

Global Architecture

The system follows a modular monolith architecture with a clear separation between frontend and backend.

- **Frontend:** React 18+ application (in `frontend/`) using Material-UI, `axios`, and functional components.
- **Backend:** FastAPI application (in `backend/`) using a strict 3-layer pattern (Router → Service → Repository), SQLAlchemy (PostgreSQL + `pgvector`), and Celery (Redis) for async tasks.
- **AI/ML:** Python-based processing pipeline using `faster-whisper` (transcription), `pyannote.audio` (diarization), `sentence-transformers` (embeddings), and LLMs (OpenAI/Ollama).

Backend Guidelines (`backend/`)

1. Modular Structure

Organize code by feature domain in `backend/app/modules/`. Each module should be self-contained:

- `router.py`: HTTP layer only — route definitions, input validation, response formatting.
- `models.py`: SQLAlchemy ORM models.
- `schemas.py`: Pydantic models for request/response validation.
- `repository.py`: Data access layer — ALL database queries must live here.
- `service.py`: Business logic, orchestration, cross-cutting concerns.

Active modules: `meetings`, `projects`, `settings`, `search`, `calendar`, `chat`, `diary`, `graph`, `users`, `admin`.

2. 3-Layer Architecture (Mandatory)

Every new endpoint or feature **must** follow the Router → Service → Repository pattern:

```
router.py → service.py → repository.py → DB
(HTTP)      (Logic)      (Queries)
```

Router Layer Rules

- Use the `_service(db)` factory pattern to instantiate services:

```
def _service(db: Session) -> MeetingService:
    return MeetingService(db)
```

```
@router.get("/{meeting_id}")
def get_meeting(meeting_id: int, db: Session = Depends(get_db)):
    return _service(db).get_meeting_or_404(meeting_id)
```

- **NEVER** put `db.query()`, business logic, or data transformations in routers.
- **NEVER** import or call repositories directly from routers (use the service layer).
- Only handle HTTP concerns: parse params, call service, format response, raise `HTTPException`.

Service Layer Rules

- Instantiate repositories in `__init__`:

```
class MeetingService:
    def __init__(self, db: Session):
        self.db = db
        self.repo = MeetingRepository(db)
        self.speaker_repo = SpeakerRepository(db)
```

- **NEVER** write `db.query()` directly — use repository methods.
- For cross-module data, import the other module's **service**, not its repository:

```
# ☒ Correct
from app.modules.settings.service import SettingsService
model_config = SettingsService(self.db).get_default_model_configuration()

# ☐ Wrong – don't reach into another module's repository
from app.modules.settings.repository import SettingsRepository
```

Repository Layer Rules

- Extend `BaseRepository` from `app.core.base.repository`:

```
class MeetingRepository(BaseRepository[Meeting, MeetingCreate,
MeetingUpdate]):
    def __init__(self, db: Session):
        super().__init__(Meeting, db)
```

- `BaseRepository` provides: `get()`, `get_or_raise()`, `get_all()`, `create()`, `update()`, `delete()`, `exists()`, `count()`.
- Add custom query methods to the repository class. All `db.query()` calls **must** be inside repository files.
- Handle `db.commit()` and `db.refresh()` within repository methods.

3. Core Processing Files

- `backend/app/tasks.py`: Celery task definitions — uses repositories (not services) since tasks run outside HTTP context.
- `backend/app/core/processing/pipeline.py`: Meeting processing pipeline — uses repositories directly.
- `backend/app/core/llm/tools.py`: LLM tool handlers — uses repositories directly.
- `backend/app/startup.py`: Application startup hooks — uses repositories directly.
- In core files, instantiate repositories directly: `MeetingRepository(db).get_by_id(id)`.

4. Database & Persistence

- **Repository Pattern**: ALWAYS use repositories for database interactions.
 - Do: `MeetingRepository(db).get_by_id(id)`
 - Don't: `db.query(Meeting).filter(...)` anywhere outside repository files.
- **No crud.py files**: All crud files have been deleted. Do NOT create new ones.
- **Migrations**: Use Alembic for schema changes. Create revisions for ANY model change:
 - `alembic revision --autogenerate -m "description"`
- **Vector Search**: Uses `pgvector` extension. Vector operations must be in `repository.py`.

5. Asynchronous Tasks

- Use **Celery** for long-running tasks (transcription, summarization, embedding computation).
- Define tasks in `backend/app/tasks.py`.
- Worker configuration in `backend/app/worker.py`.
- Tasks use repositories directly (not services) for database operations.

6. LLM & AI Integration

- Logic resides in `backend/app/core/llm/`.
- Use `LLMProvider` abstraction for model interactions (supports OpenAI and Ollama).
- **Configuration**: Managed via `backend/app/core/config.py` and `.env`.

7. Testing

- Use `pytest`.
- **Markers**: Use `@pytest.mark.unit`, `@pytest.mark.integration`, `@pytest.mark.slow`, `@pytest.mark.llm` to categorize tests.
- **Fixtures**: Defined in `backend/tests/conftest.py`.
- Test each layer independently: mock repositories in service tests, mock services in router tests.
- **Mandatory for every code change**: Add or update automated tests that cover the new/changed behavior.
- **Do not consider work complete without verification**: Run the relevant test suite(s) and ensure they pass before finishing.

Frontend Guidelines (`frontend/`)

1. Component Structure

- Functional components with Hooks only (no class components).
- Component organization:
 - `frontend/src/components/common/`: Reusable UI components (e.g., `ConfirmDialog`, `EmptyState`, `LoadingSkeleton`, `StatusChip`, `PageHeader`, `FilterBar`, `SearchInput`).
 - `frontend/src/components/features/`: Feature-specific components organized by domain (e.g., `meetings/`, `chat/`, `projects/`, `calendar/`, `diary/`, `settings/`, `kanban/`).
 - `frontend/src/components/layout/`: Layout components.
 - `frontend/src/components/upload/`: Upload-specific components.
- Use Material-UI (MUI) for styling and components.
- **Component Decomposition**: Keep components small and focused. Use container + presentational splits and custom hooks for logic.
- **Reusability**: If a UI block can be reused in multiple places, extract it to `components/common/` and pass data/behavior via props.

2. API Interaction

- **Service Layer**: Do NOT make API calls directly in components.
- Use `frontend/src/services/` modules which wrap the base `ApiClient`:
 - `meetingService.js`, `chatService.js`, `projectService.js`, `settingsService.js`, `diaryService.js`, `calendarService.js`, `searchService.js`, `graphService.js`, `speakerService.js`, `actionItemService.js`, `attachmentService.js`, `googleDriveService.js`.
- Base client is configured in `frontend/src/services/apiClient.js` (axios instance with interceptors).
- Each service exports a plain object with methods (e.g., `MeetingService.getAll()`, `MeetingService.upload()`).

3. State Management

- Use React Context for global state (`ThemeContext`).
- Use local state (`useState`, `useReducer`) for component-level data.
- Custom hooks in `frontend/src/hooks/` for reusable stateful logic.

4. Utilities & Constants

- `frontend/src/utils/`: Shared utility functions (e.g., logger).
- `frontend/src/constants/`: Application-wide constants.

Developer Workflow

1. Environment Setup:

- `backend/.env` is required (copy `example.env`).
- Prerequisites: Docker, NVIDIA GPU (optional), OpenAI Key or Ollama.

2. Run Application:

- `docker compose -f docker-compose.yml -f docker-compose.gpu.yml up --build -d` (GPU support).
- `docker compose -f docker-compose.yml up --build -d` (CPU only).

3. Running Tests:

- Backend: `pytest backend/tests` (run from root or backend dir, use conda `meetAssistant` environment). NOTE: they take quite a while, do not try to execute a new command while they are running as it results in them being killed.
- Frontend: `npm test` inside `frontend/`.
- **Mandatory:** For backend changes, run backend tests. For frontend changes, run frontend tests. For cross-stack changes, run both.
- **Pull request readiness rule:** No task is complete until tests for affected areas are executed successfully.

4. Key Anti-Patterns to Avoid:

- ✗ Creating `crud.py` files (use `repository.py` instead).
- ✗ Writing `db.query()` outside of repository files.
- ✗ Importing repositories in routers (use services).
- ✗ Importing another module's repository in a service (use that module's service).
- ✗ Making API calls directly in React components (use services).
- ✗ Putting business logic in routers.